Architecture design patterns that support performance efficiency

When you design workload architectures, you should use industry patterns that address common challenges. Patterns can help you make intentional tradeoffs within workloads and optimize for your desired outcome. They can also help mitigate risks that originate from specific problems, which can impact reliability, security, cost, and operations. If not mitigated, risks will eventually lead to performance inefficiencies. These patterns are backed by real-world experience, are designed for cloud scale and operating models, and are inherently vendor agnostic. Using well-known patterns as a way to standardize your workload design is a component of operational excellence.

Many design patterns directly support one or more architecture pillars. Design patterns that support the Performance Efficiency pillar address scalability, performance tuning, task prioritization, and the removal of bottlenecks.

The following table summarizes Architecture design patterns that support the goals of performance efficiency.

Expand table

Pattern	Summary
Asynchronous Request-Reply	Improves the responsiveness and scalability of systems by decoupling the request and reply phases of interactions for processes that don't need immediate answers. By using an asynchronous pattern, you can maximize concurrency on the server side. You can use this pattern to schedule work to be completed as capacity allows.
Backends for Frontends	Individualizes the service layer of a workload by creating separate services that are exclusive to a specific frontend interface. This separation enables you to optimize in ways that might not be possible with a shared service layer. When you handle individual clients differently, you can optimize performance for a specific client's constraints and functionality.
Bulkhead	Introduces segmentation between components to isolate the blast radius of malfunctions. This design enables each bulkhead to be individually scalable to meet the needs of the task that's encapsulated in the bulkhead.
Cache-Aside	Optimizes access to frequently read data by introducing a cache that's populated on demand. The cache is then used on subsequent requests for the same data. This pattern is especially useful with read-heavy data that doesn't

Pattern	Summary
	change often and can tolerate a certain amount of staleness. The goal of this implementation is to provide better performance in the system overall by offloading this type of data to a cache instead of sourcing it from its data store.
Choreography	Coordinates the behavior of autonomous distributed components in a workload by using decentralized, event-driven communication. This pattern can provide an alternative when performance bottlenecks occur in a centralized orchestration topology.
Circuit Breaker	Prevents continuous requests to a malfunctioning or unavailable dependency. A retry-on-error approach can lead to excessive resource utilization during dependency recovery and can also overload performance on a dependency that's attempting recovery.
Claim Check	Separates data from the messaging flow, providing a way to separately retrieve the data related to a message. This pattern improves the efficiency and performance of message publishers, subscribers, and the message bus itself when the system handles large data payloads. It works by decreasing the size of messages and ensuring that consumers retrieve payload data only if necessary and at an opportune time.
Competing Consumers	Applies distributed and concurrent processing to efficiently handle items in a queue. This model supports distributing load across all consumer nodes and dynamic scaling that's based on queue depth.
Compute Resource Consolidation	Optimizes and consolidates compute resources by increasing density. This pattern combines multiple applications or components of a workload on a shared infrastructure. This consolidation maximizes the utilization of computing resources by using spare node capacity to reduce overprovisioning. Container orchestrators are a common example. Large (vertically scaled) compute instances are often used in the resource pool for these infrastructures.
Command and Query Responsibility Segregation (CQRS)	Separates the read and write operations of an application's data model. This separation enables targeted performance and scaling optimizations for each operation's specific purpose. This design is most helpful in applications that have a high read-to-write ratio.
Deployment Stamps	Provides an approach for releasing a specific version of an application and its infrastructure as a controlled unit of deployment, based on the assumption that the same or different versions will be deployed concurrently. This pattern often aligns to the defined scale units in your workload: as additional capacity is needed beyond what a single scale unit provides, an additional deployment stamp is deployed for scaling out.
Event Sourcing	Treats state change as series of events, capturing them in an immutable, append-only log. Depending on your workload, this pattern, usually combined with CQRS, an appropriate domain design, and strategic snapshotting, can improve performance. Performance improvements are due to the atomic

Pattern	Summary
	append-only operations and the avoidance of database locking for writes and reads.
Federated Identity	Delegates trust to an identity provider that's external to the workload for managing users and providing authentication for your application. When you offload user management and authentication, you can devote application resources to other priorities.
Gatekeeper	Offloads request processing that's specifically for security and access control enforcement before and after forwarding the request to a backend node. This pattern is often used to implement throttling at a gateway level rather than implementing rate checks at the node level. Coordinating rate state among all nodes isn't inherently performant.
Gateway Aggregation	Simplifies client interactions with your workload by aggregating calls to multiple backend services in a single request. This design can incur less latency than a design in which the client establishes multiple connections. Caching is also common in aggregation implementations because it minimizes calls to backend systems.
Gateway Offloading	Offloads request processing to a gateway device before and after forwarding the request to a backend node. Adding an offloading gateway to the request process enables you to use less resources per-node because functionality is centralized at the gateway. You can optimize the implementation of the offloaded functionality independently of the application code. Offloaded platform-provided functionality is already likely to be highly performant.
Gateway Routing	Routes incoming network requests to various backend systems based on request intents, business logic, and backend availability. Gateway routing enables you to distribute traffic across nodes in your system to balance load.
Geode	Deploys systems that operate in active-active availability modes across multiple geographies. This pattern uses data replication to support the ideal that any client can connect to any geographical instance. You can use it to serve your application from a region that's closest to your distributed user base. Doing so reduces latency by eliminating long-distance traffic and because you share infrastructure only among users that are currently using the same geode.
Health Endpoint Monitoring	Provides a way to monitor the health or status of a system by exposing an endpoint that's specifically designed for that purpose. You can use these endpoints to improve load balancing by routing traffic to only nodes that are verified as healthy. With additional configuration, you can also get metrics on available node capacity.
Index Table	Optimizes data retrieval in distributed data stores by enabling clients to look up metadata so that data can be directly retrieved, avoiding the need to do full data store scans. Clients are pointed to their shard, partition, or endpoint, which can enable dynamic data partitioning for performance optimization.

Pattern	Summary
Materialized View	Uses precomputed views of data to optimize data retrieval. The materialized views store the results of complex computations or queries without requiring the database engine or client to recompute for every request. This design reduces overall resource consumption.
Priority Queue	Ensures that higher-priority items are processed and completed before lower-priority items. Separating items based on business priority enables you to focus performance efforts on the most time-sensitive work.
Publisher/Subscriber	Decouples components of an architecture by replacing direct client-to-service or client-to-services communication with communication via an intermediate message broker or event bus. The decoupling of publishers from consumers enables you to optimize the compute and code specifically for the task that the consumer needs to perform for the specific message.
Queue-Based Load Leveling	Controls the level of incoming requests or tasks by buffering them in a queue and letting the queue processor handle them at a controlled pace. This approach enables intentional design on throughput performance because the intake of requests doesn't need to correlate to the rate in which they're processed.
Scheduler Agent Supervisor	Efficiently distributes and redistributes tasks across a system based on factors that are observable in the system. This pattern uses performance and capacity metrics to detect current utilization and route tasks to an agent that has capacity. You can also use it to prioritize the execution of higher priority work over lower priority work.
Sharding	Directs load to a specific logical destination to handle a specific request, enabling colocation for optimization. When you use sharding in your scaling strategy, the data or processing is isolated to a shard, so it competes for resources only with other requests that are directed to that shard. You can also use sharding to optimize based on geography.
Sidecar	Extends the functionality of an application by encapsulating non-primary or cross-cutting tasks in a companion process that exists alongside the main application. You can move cross-cutting tasks to a single process that can scale across multiple instances of the main process, which reduces the need to deploy duplicate functionality for each instance of the application.
Static Content Hosting	Optimizes the delivery of static content to workload clients by using a hosting platform that's designed for that purpose. Offloading responsibility to an externalized host helps mitigate congestion and enables you to use your application platform only to deliver business logic.
Throttling	Imposes limits on the rate or throughput of incoming requests to a resource or component. When the system is under high demand, this pattern helps mitigate congestion that can lead to performance bottlenecks. You can also use it to proactively avoid noisy neighbor scenarios.

Pattern	Summary
Valet Key	Grants security-restricted access to a resource without using an intermediary resource to proxy the access. Doing so offloads processing as an exclusive relationship between the client and the resource without requiring an ambassador component that needs to handle all client requests in a performant way. The benefit of using this pattern is most significant when the proxy doesn't add value to the transaction.

Next steps

Review the Architecture design patterns that support the other Azure Well-Architected Framework pillars:

- Architecture design patterns that support reliability
- Architecture design patterns that support security
- Architecture design patterns that support operational excellence
- Architecture design patterns that support cost optimization

---

Last updated on 10/10/2024